

Sentiment CNN

Class Definition

```
1 class SentimentCNN(nn.Module):
```

Creates CNN model for sentiment analysis.

Constructor

```
1     def __init__(self, vocab_size, embedding_dim=256, n_filters=128,  
2                   filter_sizes=[3, 4, 5], output_dim=3, dropout=0.5):
```

Initialization with parameters:

- vocab_size: Number of unique words
- embedding_dim=256: Word vector size
- n_filters=128: Number of filters per kernel size
- filter_sizes=[3, 4, 5]: Kernel sizes for convolutions
- output_dim=3: Sentiment classes
- dropout=0.5: Dropout rate

```
1     super().__init__()
```

Initializes parent class.

```
1     self.embedding = nn.Embedding(vocab_size, embedding_dim,  
                                   padding_idx=0)
```

Creates embedding layer.

```
1         # Convolutions 1D avec differentes tailles de filtres  
2         self.convs = nn.ModuleList([  
3             nn.Conv1d(  
4                 in_channels=embedding_dim,  
5                 out_channels=n_filters,  
6                 kernel_size=fs  
7             )  
8             for fs in filter_sizes  
9         ])
```

Creates list of 1D convolutional layers with different kernel sizes:

- 3 separate Conv1D layers (for kernels 3, 4, 5)
- Each has 256 input channels and 128 output channels
- Captures n-grams of different lengths

```
1 self.dropout = nn.Dropout(dropout)
```

Creates dropout layer.

```
1 self.fc = nn.Linear(len(filter_sizes) * n_filters, output_dim)
```

Creates output layer. Input size: $3 \times 128 = 384$ (concatenated features from all filters).

```
1 # Initialisation des poids (Xavier pour stabilite)
2 self._init_weights()
```

Calls custom weight initialization method.

Weight Initialization Method

```
1 def _init_weights(self):
```

Defines custom weight initialization.

```
1 for conv in self.convs:
2     nn.init.xavier_uniform_(conv.weight)
```

Initializes each convolutional layer's weights using Xavier uniform distribution for training stability.

```
1 nn.init.xavier_uniform_(self.fc.weight)
```

Initializes fully connected layer weights using Xavier uniform.

Forward Method

```
1 def forward(self, input_ids, attention_mask=None):
```

Defines forward pass. `attention_mask` is optional and not used in this model.

```
1 # Embedding
2 embedded = self.embedding(input_ids)
3 embedded = self.dropout(embedded)
```

Converts indices to embeddings and applies dropout. Shape: (batch, sequence, 256).

```
1 # Permutation pour Conv1d
2 embedded = embedded.permute(0, 2, 1)
```

Transposes dimensions for Conv1D requirement. Shape: (batch, sequence, 256) $\rightarrow$ (batch, 256, sequence). Conv1D expects (batch, channels, length).

```
1 # Appliquer chaque convolution + ReLU + Max pooling global
2 convded = [
3     F.relu(conv(embedded))
4     for conv in self.convs
5 ]
```

Applies each convolution followed by ReLU activation. Creates list of 3 tensors, each shape: (batch, 128, variable_length).

```
1      # Max pooling global sur la dimension temporelle
2      pooled = [
3          F.max_pool1d(conv, conv.shape[2]).squeeze(2)
4          for conv in convded
5      ]
```

Applies global max pooling over entire sequence length for each convolution. Extracts most important feature from each filter. Shape per tensor: (batch, 128, 1) → (batch, 128).

```
1      # Concatener les features de tous les filtres
2      cat = self.dropout(torch.cat(pooled, dim=1))
```

Concatenates features from all filter sizes and applies dropout. Shape: (batch, 384).

```
1      # Classification
2      return self.fc(cat)
```

Applies linear layer to produce 3 logits.

Model Instantiation

```
1 model = SentimentCNN(
2     vocab_size=len(vocab),
3     embedding_dim=256,
4     n_filters=128,
5     filter_sizes=[3, 4, 5],
6     output_dim=3,
7     dropout=0.5
8 ).to(device)
```

Creates model instance and moves to device.

```
1 print(f" Modele CNN 1D initialise :")
2 print(f"   - Embedding dim : 256")
3 print(f"   - Filtres          : 128 par taille")
4 print(f"   - Tailles            : {3}, {4}, {5}      capture les n-grammes
5     critiques")
6 print(f"   - Parametres        : {sum(p.numel() for p in model.parameters())
7     },}")
```

Prints model configuration and parameter count.

Tools Used

PyTorch Core (torch)

- torch.cat: Concatenates tensors

Neural Network Module (nn)

- nn.Module: Base class

- `nn.Embedding`: Word embeddings
- `nn.ModuleList`: Container for multiple modules
- `nn.Conv1d`: 1D convolutional layer
- `nn.Linear`: Fully connected layer
- `nn.Dropout`: Regularization
- `nn.init.xavier_uniform_`: Weight initialization method

Functional Interface (F)

- `F.relu`: Activation function
- `F.max_pool1d`: Max pooling operation

Key Architecture Features

- **Multiple filter sizes**: Captures different n-gram patterns (3-word, 4-word, 5-word phrases)
- **Global max pooling**: Extracts single most important feature from each filter
- **Feature concatenation**: Combines information from all filter sizes
- **No recurrence**: Processes entire sequence in parallel (faster than LSTM)